CARAT: Class-Aware Robust Aggregation for Model-Poisoning-Resilient Federated Learning

Anonymous Author(s)
Affiliation
Address
email

Abstract

Federated learning servers aggregate client updates without seeing client data, which leaves training exposed to model-poisoning attacks. Most robust aggregators rely on update geometry, coordinate-wise robust statistics, or alignment with a single trusted reference direction. These signals weaken when benign clients are heterogeneous and adaptive attackers stay close to the benign update manifold. We propose **CARAT**, a class-aware robust aggregation rule built on *hidden task certificates*. Given a small labeled server-side reference set, CARAT samples multiple hidden, class-balanced micro-tasks and scores each client update by the loss reduction it achieves across them. CARAT combines these task-conditioned scores with robust clipping, common-radius evaluation, coordinate-rank anomaly penalties, and capped-simplex weight optimization with temporal regularization. The result is an aggregator that favors updates that remain useful across diverse semantic slices while limiting the influence of structurally extreme clients. On the completed CIFAR-100/ResNet18 non-i.i.d. benchmark block with 20% malicious clients, CARAT ranks first against six baselines across ALIE, FangAttack, MinMax, and MinSum under $\alpha \in \{1, 0.5\}$. We frame CARAT specifically as a defense against untargeted model poisoning under heterogeneous client data, with an evaluation protocol emphasizing class-rich benchmarks, matched trusted-data budgets, and class-sensitive metrics.

1 Introduction

Federated learning (FL) lets many clients train a shared model without centralizing raw data [1–3]. This decentralization is attractive in privacy- and governance-sensitive settings, but it leaves the server in an asymmetric position: the global model must be updated from client messages that the server cannot directly verify. The aggregation rule therefore becomes a security-critical part of the FL system. Malicious clients can slow convergence or distort the decision boundary through model-poisoning updates that remain difficult to distinguish from benign client drift [4–7].

The central difficulty is that benign heterogeneity and malicious behavior can look similar from the server's perspective. Under non-i.i.d. partitions, benign clients can differ substantially in update direction, norm, sparsity, and class emphasis [8–11]. An unusual update may therefore reflect either a harmful client or a legitimately different local distribution. This ambiguity weakens defenses based on coarse notions of centrality. Pairwise-distance methods can mistake benign dispersion for adversarial deviation, coordinate-wise robust statistics discard cross-coordinate structure, and trusted-anchor approaches compress all auxiliary supervision into a single global reference direction [12–19]. Such signals are often too coarse to expose attacks whose effect is concentrated on a subset of classes or semantic regions.

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

CARAT addresses this problem by shifting validation from geometric similarity to task-conditioned utility. The server maintains a small labeled reference set and repeatedly instantiates hidden, class-balanced micro-tasks from it. Each client update is scored by the loss reduction it induces across these tasks, producing a certificate matrix that measures whether the update is broadly useful across semantic slices of the problem instead of merely close to the geometric center of the client population. CARAT then combines this signal with robust clipping, common-radius evaluation, coordinate-rank anomaly penalties, and capped-simplex weight optimization with temporal regularization.

This design targets FL deployments in which a modest server-side audit set is realistic, such as cross-silo or institutionally managed settings. It is not a fully data-free defense. Within that setting, CARAT aims to preserve benign diversity while reducing the influence of updates that are semantically harmful or structurally extreme. Our main contributions are as follows.

- We propose CARAT, a class-aware robust aggregation rule that validates client updates through hidden-task certificates constructed from server-side labeled reference data, replacing one-shot geometric trust signals with multi-task utility checks.
- We formulate aggregation as a capped-simplex optimization problem that combines worst-task utility across hidden tasks, coordinate-rank anomaly suppression, and temporal regularization, yielding a robust weighting mechanism compatible with heterogeneous FL.
- We report a completed CIFAR-100/ResNet18 non-i.i.d. comparison in which CARAT ranks first against six baselines across four model-poisoning attacks under $\alpha \in \{1, 0.5\}$, while identifying the remaining $\alpha = 0.1$ and seed-completion gaps explicitly.

2 Related Work

Model-poisoning attacks. Modern FL attacks are designed to remain effective under server-side defenses. Early work already showed that even constrained adversaries can poison the global model while remaining relatively stealthy [4–6]. In particular, attacks such as ALIE, MinMax, and MinSum craft malicious updates that evade simple geometric or statistical filters while still harming convergence or accuracy [6, 7]. Surveys make the broader point explicit: once the server cannot inspect raw client data, model poisoning becomes a systems concern rather than an edge case [3, 20].

Robust aggregation under heterogeneity. Classical Byzantine-robust aggregation rules infer benignness from centrality. Krum and related geometry-based methods favor updates that remain close to the client population under pairwise distances [12, 13], while coordinate-wise median and trimmed mean suppress marginal extremes independently [14]. Later variants extend this family through suspicion scoring, regularized aggregation, coding-based redundancy, and geometric-median aggregation [15–18, 21]. These methods are attractive because they require little server-side prior knowledge. Under heterogeneous data, however, they can still confuse legitimate benign dispersion with malicious deviation. This limitation has motivated methods that address heterogeneity more directly through optimization stabilization, resampling, bucketing, local-SGD analysis, gradient splitting, nearest-neighbor mixing, label-skew-aware aggregation, and entropy-based public-data guidance [9–11, 22–28].

Trusted-data and history-based defenses. A third line of work augments aggregation with trusted data, history, or auxiliary diagnostics. In collaborative learning, AUROR anticipated update screening for poisoning resistance [29]. In FL, FLTrust introduces a trusted server anchor, while FoolsGold and FLDetector rely on historical similarity diagnostics [19, 30, 31]. Recent work also highlights the cost of robustness itself, including clipping design and the tradeoff between Byzantine resilience and clean accuracy [32, 33]. CARAT is closest to this broader defense-and-detection family, but it uses a different validation principle: instead of reducing trust to a single anchor direction, geometric similarity, or post-hoc clustering signal, it asks whether a client update remains useful across multiple hidden semantic sub-tasks.

3 Method

3.1 Formulation and Assumptions

We consider synchronous FL with n clients and a global parameter vector $w_r \in \mathbb{R}^d$ at round r. At each round, client i returns a message u_i^r, which the server maps to a vector-form update $\Delta_i^r \in \mathbb{R}^d$. For FedAvg, Δ_i^r is the client model delta relative to the current global model; for gradient-style optimizers, it is the client update itself. The server must aggregate $\{\Delta_i^r\}_{i=1}^n$ into a single global step while a fraction ρ of clients may be malicious and coordinated.

CARAT assumes the server has access to labeled reference data $\mathcal{D}^{\text{ref}}$. From this data it constructs a probe pool $\mathcal{P}$ and repeatedly samples hidden, class-balanced validation tasks. This assumption is stronger than fully data-free robust aggregation and should be treated as part of the threat model, not as an incidental implementation detail. CARAT uses the auxiliary data not to define a single trusted direction, but to generate multiple hidden semantic checks against which candidate client updates can be evaluated.

The objective is to assign aggregation weights that preserve benign diversity while suppressing updates that are semantically harmful or structurally anomalous. This distinction matters in non-i.i.d. FL, where benign clients may differ substantially in direction, norm, and class emphasis. Geometric dispersion alone is therefore insufficient. CARAT divides each round into three stages: common-geometry evaluation, task-aware and structural scoring, and robust weight optimization. Evaluation is normalized across clients, while the final server step remains conservative because it is computed from clipped updates.

3.2 Method Details

At round r, CARAT first converts client messages into vector-form deltas and computes their norms. Before any semantic evaluation, the server robustly clips client updates so that a small number of large-magnitude messages cannot dominate the comparison stage. Let $\|\Delta_i\|_2$ denote the norm of client i's update. In our implementation, the clipping threshold τ_r is estimated by the MAD-based rule

$$\tau_r = \text{median}(\{\|\Delta_i\|_2\}) + k \cdot 1.4826 \cdot \text{MAD}(\{\|\Delta_i\|_2\}), \tag{1}$$

and the clipped update is

$$\bar{\Delta}_i = \min\left(1, \frac{\tau_r}{\|\Delta_i\|_2 + \varepsilon}\right) \Delta_i. \tag{2}$$

CARAT then rescales each clipped update to a common evaluation radius q_r:

$$\tilde{\Delta}_i = q_r \frac{\bar{\Delta}_i}{\|\bar{\Delta}_i\|_2 + \varepsilon}, \tag{3}$$

where q_r is chosen as a quantile of the positive clipped norms. This normalization removes pure scale advantages during hidden-task comparison. The normalized vectors are used only for evaluation; the final model update is still computed from the clipped vectors. CARAT uses normalization to create a fair scoring geometry, not to define the actual optimization step taken by the server.

After preprocessing, CARAT samples T hidden tasks from the probe pool $\mathcal{P}$, with each task constructed to be approximately class-balanced. This is what makes the method class-aware. A malicious update can remain globally plausible while degrading only a subset of classes or semantic regions. Repeated balanced hidden tasks make these localized failures harder to hide. For each hidden task t, let $\ell_t(w)$ denote the average cross-entropy loss of model w on that task. CARAT defines a *hidden-task certificate*

$$c_{i,t} = \ell_t(w_r) - \ell_t(w_r + \tilde{\Delta}_i). \tag{4}$$

Here, "certificate" denotes an internal task-level utility score, not a formal certified-robustness guarantee. Positive values indicate that client i improves task t relative to the current global model.

Stacking these scores yields a certificate matrix $C \in \mathbb{R}^{n \times T}$. Because different tasks may have different intrinsic difficulty and baseline loss scale, CARAT robustly standardizes each column of C before optimization. This preserves within-task ordering while making scores comparable across hidden slices. The resulting matrix gives the server a richer signal than client-to-client similarity

alone: a client should be trusted because it remains broadly useful across hidden tasks, not merely because it lies near the geometric center of the client population.

Task utility alone is insufficient, because a malicious client may overfit the probe tasks or align with part of the semantic signal while remaining structurally extreme in parameter space. CARAT therefore augments hidden-task certificates with a coordinate-rank anomaly score. For a sampled coordinate set $S \subseteq \{1, \ldots, d\}$, it ranks clients coordinate-wise and measures the average normalized distance from the center rank:

$$r_i^{(S)} = \frac{1}{|S|} \sum_{j \in S} \left| \frac{\mathrm{rank}_{i,j}}{n-1} - \frac{1}{2} \right|. \tag{5}$$

CARAT repeats this computation over several random coordinate subsets and aggregates the resulting values robustly. A client receives a large anomaly score if its update remains persistently extreme across many coordinates rather than on a few isolated entries. After robust standardization, only positive deviations are retained as penalties. The signal is intentionally asymmetric: suspicious extremeness is penalized, whereas unusual centrality is not explicitly rewarded. Together, the certificate matrix and rank penalty separate two notions that are often conflated in robust aggregation: whether an update is useful and whether it is structurally atypical.

3.3 Weight Optimization and Aggregation

CARAT maps these signals to aggregation weights by solving a constrained optimization problem on the capped simplex

$$\mathcal{A}(\hat{\rho}) = \left\{ \alpha \in \mathbb{R}^n : \alpha_i \geq 0,\ \sum_{i=1}^{n} \alpha_i = 1,\ \alpha_i \leq \frac{1}{(1-\hat{\rho})n} \right\}, \tag{6}$$

where $\hat{\rho}$ is an estimate of the attacker fraction. The cap prevents any single client from dominating the aggregate and encodes a conservative trust budget at the weight level: even when a client appears useful, it cannot absorb arbitrary influence within one round.

Let $r \in \mathbb{R}^n$ denote the positive rank penalties and let α^{prev} be the previous round's weights. CARAT solves

$$\max_{\alpha \in \mathcal{A}(\hat{\rho})} \ \operatorname*{softmin}_{\beta}(C^\top \alpha) - \lambda_r r^\top \alpha - \lambda_p \|\alpha - \alpha^{\mathrm{prev}}\|_2^2, \tag{7}$$

where

$$\operatorname*{softmin}_{\beta}(s_1, \ldots, s_T) = -\frac{1}{\beta} \log \left(\frac{1}{T} \sum_{t=1}^{T} e^{-\beta s_t} \right). \tag{8}$$

The first term is a smooth approximation to worst-task utility: large β places more emphasis on the weakest hidden tasks, while finite β preserves optimization stability. The second term penalizes structurally extreme clients, and the third term regularizes the solution toward the previous round to reduce unnecessary oscillation. Eq. (7) therefore implements a simple decision rule: a client should retain influence only if it remains useful across hidden tasks and does not repeatedly appear as an outlier in rank space.

After optimization, CARAT aggregates the clipped updates:

$$\Delta_{\mathrm{agg}} = \sum_{i=1}^{n} \alpha_i \bar{\Delta}_i. \tag{9}$$

This is the final server step. The aggregate inherits the robustness benefits of clipping while using weights learned from the richer evaluation geometry defined above. In practice, the main additional computation comes from evaluating n candidate updates over T hidden tasks and computing rank penalties on coordinate subsamples. Probe-model forward passes dominate, while the ranking and optimization stages remain tunable through explicit budget parameters.

A compact round-level algorithm summary is provided in Appendix A. Formal properties of the normalization step, capped-simplex feasibility, softmin behavior, temporal regularization, and per-round complexity are deferred to Appendix B.

4 Experiments

We evaluate CARAT on class-rich federated benchmarks to test whether hidden-task certificates improve robustness beyond geometry-only and single-anchor defenses while preserving clean utility under heterogeneous client data. The study is organized around three questions: how much clean-utility cost CARAT incurs without attack, how well it resists untargeted model poisoning as heterogeneity increases, and what robustness is obtained per unit of trusted-data budget and runtime overhead. The full benchmark matrix, appendix-only stress tests, and ablation scope are reported in Appendix C.

4.1 Experimental Setup

Benchmarks. The primary benchmark is CIFAR-100 with ResNet18 [34]. CIFAR-100 offers a class-rich label space in which hidden semantic slices are meaningful and class-selective failures are easier to expose than on low-cardinality datasets. We complement it with TinyImageNet, again using ResNet18, to test whether the method scales to a larger class vocabulary while keeping the backbone fixed. To anchor the method in a more application-oriented setting, we also include CHMNIST with the same backbone in the appendix, where class-selective degradation has a clearer domain interpretation. CIFAR-10 is retained only as an appendix sanity check rather than a primary source of evidence, because a ten-class benchmark is too coarse to serve as the main validation vehicle for a class-aware defense. Keeping the model family fixed across visual datasets helps attribute observed differences to aggregation rather than to backbone changes.

Threat models. For untargeted model poisoning, the main threat models are ALIE [7], FangAttack [5], MinMax [6], and MinSum [6]. These attacks represent stealthy optimization-based adversaries that try to stay near the benign update manifold while degrading global learning, together with a geometry-aware attack that is especially informative for defenses based on Krum-style screening. We do not treat data poisoning or backdoor attacks as part of CARAT's claimed scope in the current paper.

Baselines. For the completed CIFAR-100 comparison, the primary baselines are Mean, NormClipping, MultiKrum [12], FLTrust [19], FLDetector [31], TriGuardFL, and CARAT. This set mirrors the strongest directly reusable baseline block in the existing experiment library and spans plain averaging, norm clipping, geometry-based filtering, trusted-data anchoring, history-based detection, and class-aware hidden-task validation. TrimmedMean [14] is retained for the smaller i.i.d. diagnostic runs, but it is not part of the completed non-i.i.d. CIFAR-100 benchmark block used in Table 2.

Implementation details. Across the primary completed comparison, we use 20 clients with full participation, 200 global rounds, 4 local epochs, and learning rate 0.05. To probe heterogeneity, we evaluate Dirichlet non-i.i.d. regimes with $\alpha \in \{1, 0.5\}$ in the completed table; the more severe $\alpha = 0.1$ CARAT block is still pending and is therefore not reported as a main result. The adversarial setting uses an attacker ratio of 20%, corresponding to 4 malicious clients. Clean-reference experiments are reported separately from attacked experiments instead of being inserted as attack rows. Unless otherwise stated, baseline means and standard deviations are computed over 5 seeds, while completed CARAT rows currently use 1–3 seeds depending on which incremental jobs have finished.

Metrics and fairness. Because CARAT is explicitly class-aware, the evaluation must report more than aggregate clean accuracy. For clean utility, we report test accuracy and test loss under no attack. For robustness, we additionally report macro-average accuracy and worst-class accuracy on class-rich benchmarks so that localized failures remain visible. Fairness requires explicit control of server-side supervision: CARAT and FLTrust are compared under a matched labeled-data budget, and the reference budget is parameterized per class rather than fixed as a dataset-independent total pool size.

4.2 Main Evaluation Comparisons

Clean utility. We first report no-attack performance to make the cost of robustness visible. These comparisons focus on test accuracy and test loss, with macro and worst-class accuracy added on class-rich benchmarks when aggregate accuracy alone could mask concentrated degradation.

Table 1: Experimental setup for evaluating CARAT.

Component	Design choice
Primary benchmark	CIFAR-100 with ResNet18
Supplementary benchmarks	TinyImageNet with ResNet18; CHMNIST with ResNet18 (appendix); CIFAR-10 only as appendix sanity check
Clients and participation	20 total clients with full participation each round
Optimization schedule	200 communication rounds, 4 local epochs, learning rate 0.05
Heterogeneity settings	Dirichlet non-i.i.d. with $\alpha \in \{1, 0.5\}$ completed; $\alpha = 0.1$ pending for CARAT
Main adversarial fraction	20% malicious clients
Stress adversarial fraction	40% malicious clients (appendix)
Main untargeted attacks	ALIE, FangAttack, MinMax, MinSum
Untargeted baselines	Mean, NormClipping, MultiKrum, FLTrust, FLDetector, TriGuardFL, CARAT
Primary reported metrics	Final test accuracy, final test loss, average rank, runtime per round where available
Fairness controls	Matched labeled server-reference budget between CARAT and FLTrust; clean-reference runs reported separately from attacked runs
Repetitions	Target of 3–5 seeds per configuration; current completed CARAT rows use 1–3 seeds where noted

Untargeted poisoning. We then compare CARAT against Mean, NormClipping, MultiKrum, FLTrust, FLDetector, and TriGuardFL under ALIE, FangAttack, MinMax, and MinSum on non-i.i.d. partitions. The main question is whether task-conditioned validation remains reliable when benign clients are dispersed and adaptive attackers stay close to the benign update manifold.

4.3 Ablations, Stress Tests, and Overhead

Ablations. We ablate the components that correspond directly to CARAT's mechanism: hidden-task count, reference budget per class, rank-penalty weight, temporal-prior weight, clipping controls, and common-radius evaluation. These studies isolate whether the observed robustness comes from class-balanced task validation, structural anomaly suppression, or simply from access to more trusted data.

Stress tests. Appendix-only stress tests raise the attacker fraction to 40%, complete the more severe $\alpha = 0.1$ heterogeneity block, and vary the client count or class vocabulary. Their role is to probe failure modes and scaling behavior rather than to replace the main-text comparisons.

Overhead analysis. Since Proposition B.7 identifies probe-model evaluation as the main extra cost, we report runtime per round, total defense overhead, and trusted-data budget sensitivity as first-class evaluation outcomes rather than as implementation footnotes.

4.4 Current Completed Results

The strongest completed block in the current experimental artifact is a direct non-i.i.d. CIFAR-100/ResNet18 comparison with 20 clients, 20% malicious clients, 200 rounds, and four untargeted model-poisoning attacks. Table 2 reports final test accuracy for this matched protocol. All entries are computed only from runs with completion markers; partial curves and failed jobs are excluded from the means. Baseline rows use five completed seeds in each cell. CARAT currently uses the completed incremental runs available in the artifact, so the CARAT cells use one to three seeds as indicated in the caption.

Table 2 changes the empirical story from an implementation check to a matched robustness comparison. CARAT obtains the best average rank in both completed heterogeneity regimes and is the top method in every completed attack column. Under $\alpha = 1$, CARAT improves over the strongest non-CARAT result by 2.95–3.62 percentage points across the four attacks. Under the more heterogeneous $\alpha = 0.5$ setting, CARAT improves by 2.91–4.64 points in the three cells with three

Table 2: Final test accuracy (%) under four model-poisoning attacks on CIFAR-100/ResNet18 with 20% malicious clients and non-i.i.d. client partitions. Higher is better. Baseline cells use five completed seeds. CARAT cells use completed incremental runs: $n = 2$ for $\alpha = 1$ ALIE, $n = 1$ for $\alpha = 0.5$ MinSum, and $n = 3$ otherwise.

Dirichlet α	Defense	ALIE	FangAttack	MinMax	MinSum	Avg. rank
1.0	Mean	49.43 ± 0.39	48.86 ± 0.70	50.49 ± 0.59	50.50 ± 0.49	3.0
1.0	NormClipping	25.27 ± 0.70	22.67 ± 0.77	24.86 ± 0.65	24.74 ± 0.63	7.0
1.0	MultiKrum	39.59 ± 0.74	37.18 ± 0.83	48.69 ± 0.84	47.71 ± 1.12	4.8
1.0	FLTrust	33.26 ± 0.63	33.81 ± 0.99	33.61 ± 0.86	33.85 ± 0.78	5.8
1.0	FLDetector	47.96 ± 0.58	28.54 ± 4.42	50.58 ± 0.76	50.47 ± 0.27	4.0
1.0	TriGuardFL	49.44 ± 0.56	51.01 ± 0.93	50.36 ± 0.59	50.65 ± 0.52	2.5
1.0	**CARAT**	**52.92±1.71**	**53.96±0.86**	**54.20±1.39**	**54.23±1.73**	**1.0**
0.5	Mean	47.62 ± 0.73	47.53 ± 1.01	48.88 ± 0.32	48.88 ± 0.52	3.2
0.5	NormClipping	23.73 ± 0.53	20.64 ± 1.41	23.28 ± 0.40	22.92 ± 0.50	7.0
0.5	MultiKrum	37.00 ± 0.81	33.05 ± 1.20	46.59 ± 1.09	46.45 ± 0.80	4.8
0.5	FLTrust	31.72 ± 0.51	32.20 ± 0.47	32.21 ± 0.68	32.08 ± 0.46	5.8
0.5	FLDetector	45.30 ± 0.75	27.77 ± 1.25	48.85 ± 0.53	49.01 ± 0.69	4.2
0.5	TriGuardFL	47.81 ± 0.63	49.67 ± 0.60	49.12 ± 0.68	49.16 ± 0.81	2.0
0.5	**CARAT**	**51.89±0.56**	**52.58±1.30**	**53.76±0.36**	**54.81**	**1.0**

completed CARAT seeds; the MinSum cell is currently a single completed seed and should be treated as provisional until the remaining jobs finish. FangAttack is especially informative: FLDetector drops to 28.54% and 27.77% under $\alpha = 1$ and $\alpha = 0.5$, respectively, while CARAT remains at 53.96% and 52.58%. This supports the main design claim that hidden-task validation can preserve useful heterogeneous updates while rejecting semantically harmful ones that evade or destabilize geometry-based screening.

Figure 1 gives the corresponding round-wise view for FangAttack, the attack that most sharply separates the defenses in Table 2. CARAT converges more slowly than the clean Mean reference in the early phase, but it continues improving after the geometry- and history-based defenses flatten or degrade. By the end of training, CARAT is the only attacked defense that reaches or exceeds the clean Mean reference in both completed heterogeneity regimes.

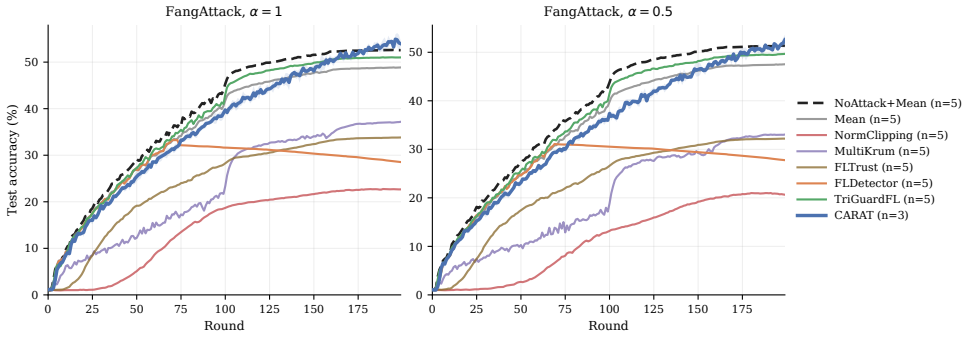

Figure 1: Evolution of test accuracy on CIFAR-100/ResNet18 under FangAttack with 20% malicious clients and non-i.i.d. data. Curves show the mean over completed seeds; the CARAT band shows one standard deviation. The dashed black curve is the no-attack Mean reference under the same client partition protocol.

The corresponding final-loss summaries in Table 3 follow the same ranking pattern: CARAT has average rank 1.0 under both $\alpha = 1$ and $\alpha = 0.5$. The completed clean Mean references are $52.57 \pm 0.41\%$ for $\alpha = 1$ and $51.32 \pm 0.85\%$ for $\alpha = 0.5$, which places CARAT close to the clean-utility regime even under attack. The remaining empirical gaps are therefore specific rather than structural: the $\alpha = 0.1$ CARAT block needs to finish, and the one-seed $\alpha = 0.5$ MinSum entry needs the remaining repetitions before it can be presented as a final multi-seed result.

Figure 2 gives a complementary learning-curve diagnostic from an earlier i.i.d. ALIE verification run. In that run, CARAT lags behind Mean in the early rounds and remains lower at round 170, but it stays stable across seeds instead of diverging under attack. The right panel shows the main practical tradeoff of the current implementation: CARAT pays roughly a $3.5\times$ to $4\times$ round-time premium for repeated hidden-task evaluation.

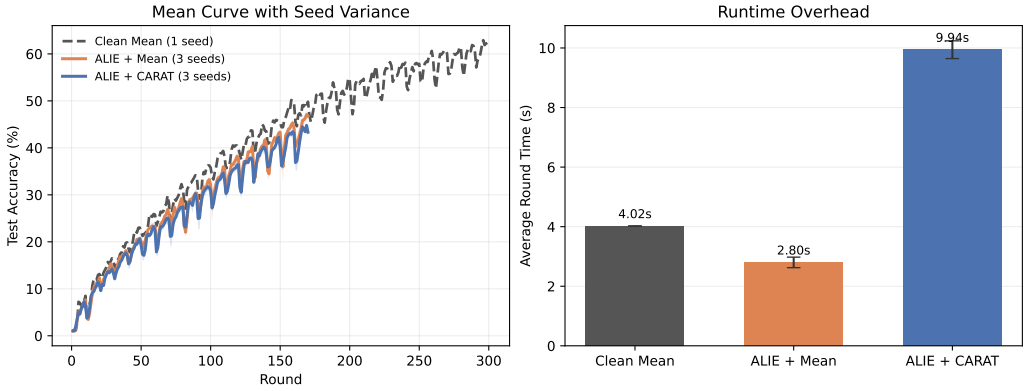

Figure 2: Verification-scale local results on CIFAR-100 under the ALIE model-poisoning attack. Left: round-wise test accuracy, where solid curves show the mean over three seeds and shaded bands denote one standard deviation; the dashed clean Mean curve is a single-seed no-attack reference. Right: average per-round wall-clock time with seed-level error bars, showing the defense overhead of repeated hidden-task evaluation.

5 Conclusion

We presented CARAT, a class-aware robust aggregation rule that validates client updates through hidden task certificates rather than relying only on geometric centrality or a single trusted reference direction. By scoring updates across multiple hidden, class-balanced semantic slices and combining those scores with rank-space anomaly suppression and capped-simplex weight optimization, CARAT is designed to preserve benign heterogeneity while reducing the influence of semantically harmful or structurally extreme clients.

The completed CIFAR-100/ResNet18 non-i.i.d. experiments support this design choice: CARAT achieves the best average rank against Mean, NormClipping, MultiKrum, FLTrust, FLDetector, and TriGuardFL across ALIE, FangAttack, MinMax, and MinSum for $\alpha \in \{1, 0.5\}$. The evidence is strongest as a heterogeneous untargeted-poisoning result rather than as a universal poisoning claim. The method is most appropriate in FL deployments where the server can maintain a modest labeled audit set and where class-selective failures matter under heterogeneous client drift. The remaining work is to complete the severe $\alpha = 0.1$ block, finish the remaining CARAT seeds, and expand the overhead and trusted-budget sensitivity studies.

References

[1] H. Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. Communication-efficient learning of deep networks from decentralized data. In *Artificial Intelligence and Statistics*, 2017.

[2] Kallista A. Bonawitz, Vladimir Ivanov, Ben Kreuter, Antonio Marcedone, H. Brendan McMahan, Sarvar Patel, Daniel Ramage, Aaron Segal, and Karn Seth. Practical secure aggregation for privacy-preserving machine learning. In *ACM SIGSAC Conference on Computer and Communications Security*, pages 1175–1191, 2017.

[3] Peter Kairouz, H. Brendan McMahan, Brendan Avent, Aurelien Bellet, Mehdi Bennis, Arjun Nitin Bhagoji, Kallista Bonawitz, Zachary Charles, Graham Cormode, Rachel Cummings, et al. *Advances and Open Problems in Federated Learning*. Now Publishers, 2021.

[4] Arjun Nitin Bhagoji, Supriyo Chakraborty, Prateek Mittal, and Seraphin Calo. Analyzing federated learning through an adversarial lens. In *International Conference on Machine Learning*, pages 634–643, 2019.

[5] Minghong Fang, Xiaoyu Cao, Jinyuan Jia, and Neil Zhenqiang Gong. Local model poisoning attacks to Byzantine-robust federated learning. In *USENIX Security Symposium*, 2020.

[6] Virat Shejwalkar and Amir Houmansadr. Manipulating the Byzantine: Optimizing model poisoning attacks and defenses for federated learning. In *Network and Distributed System Security Symposium*, 2021.

[7] Gilad Baruch, Moran Baruch, and Yoav Goldberg. A little is enough: Circumventing defenses for distributed learning. In *Advances in Neural Information Processing Systems*, 2019.

[8] Yue Zhao, Meng Li, Liangzhen Lai, Naveen Suda, et al. Federated learning with non-IID data. *arXiv preprint arXiv:1806.00582*, 2018.

[9] Tian Li, Anit Kumar Sahu, Manzil Zaheer, Maziar Sanjabi, Ameet Talwalkar, and Virginia Smith. Federated optimization in heterogeneous networks. In *Proceedings of Machine Learning and Systems*, 2020.

[10] Sai Praneeth Karimireddy, Satyen Kale, Mehryar Mohri, Sashank Reddi, Sebastian Stich, and Ananda Theertha Suresh. SCAFFOLD: Stochastic controlled averaging for federated learning. In *International Conference on Machine Learning*, pages 5132–5143, 2020.

[11] Hangyu Zhu, Jinjin Xu, Shiqing Liu, and Yaochu Jin. Federated learning on non-IID data: A survey. *Neurocomputing*, 465:371–390, 2021.

[12] Peva Blanchard, El Mahdi El Mhamdi, Rachid Guerraoui, and Julien Stainer. Machine learning with adversaries: Byzantine tolerant gradient descent. In *Advances in Neural Information Processing Systems*, 2017.

[13] El Mahdi El Mhamdi, Rachid Guerraoui, and Sébastien Rouault. The hidden vulnerability of distributed learning in Byzantium. In *International Conference on Machine Learning*, 2018.

[14] Dong Yin, Yudong Chen, Ramchandran Kannan, and Peter Bartlett. Byzantine-robust distributed learning: Towards optimal statistical rates. In *International Conference on Machine Learning*, 2018.

[15] Cong Xie, Sanmi Koyejo, and Indranil Gupta. Zeno: Distributed stochastic gradient descent with suspicion-based fault-tolerance. In *International Conference on Machine Learning*, pages 6893–6901, 2019.

[16] Lingjiao Chen, Hongyi Wang, Zachary Charles, and Dimitris Papailiopoulos. DRACO: Byzantine-resilient distributed training via redundant gradients. In *International Conference on Machine Learning*, pages 903–912, 2018.

[17] Liping Li, Wei Xu, Tianyi Chen, Georgios B. Giannakis, and Qing Ling. RSA: Byzantine-robust stochastic aggregation methods for distributed learning from heterogeneous datasets. In *AAAI Conference on Artificial Intelligence*, pages 1544–1551, 2019.

[18] Krishna Pillutla, Sham M. Kakade, and Zaid Harchaoui. Robust aggregation for federated learning. *IEEE Transactions on Signal Processing*, 70:1142–1154, 2022.

[19] Xiaoyu Cao, Minghong Fang, Jia Liu, Neil Zhenqiang Gong, and Yixin Gong. FLTrust: Byzantine-robust federated learning via trust bootstrapping. In *Network and Distributed System Security Symposium*, 2021.

[20] Pengrui Liu, Xiangrui Xu, and Wei Wang. Threats, attacks and defenses to federated learning: Issues, taxonomy and perspectives. *Cybersecurity*, 5(4), 2022.

[21] Dong Yin, Yudong Chen, Ramchandran Kannan, and Peter Bartlett. Defending against saddle point attack in byzantine-robust distributed learning. In *International Conference on Machine Learning*, pages 7074–7084, 2019.

[22] Lie He, Sai Praneeth Karimireddy, and Martin Jaggi. Byzantine-robust learning on heterogeneous datasets via resampling. In *International Conference on Learning Representations*, 2021.

[23] Sai Praneeth Karimireddy, Lie He, and Martin Jaggi. Byzantine-robust learning on heterogeneous datasets via bucketing. In *International Conference on Learning Representations*, 2022.

[24] Deepesh Data and Suhas Diggavi. Byzantine-resilient high-dimensional SGD with local iterations on heterogeneous data. In *International Conference on Machine Learning*, pages 2478–2488, 2021.

[25] Yuchen Liu, Chen Chen, Lingjuan Lyu, Fangzhao Wu, Sai Wu, and Gang Chen. Byzantine-robust learning on heterogeneous data via gradient splitting. In *International Conference on Machine Learning*, pages 21404–21425, 2023.

[26] Youssef Allouah, Sadegh Farhadkhani, Rachid Guerraoui, Nirupam Gupta, Rafael Pinot, and John Stephan. Fixing by mixing: A recipe for optimal byzantine ML under heterogeneity. In *International Conference on Artificial Intelligence and Statistics*, pages 1232–1300, 2023.

[27] Wenxuan Bao, Jun Wu, and Jingrui He. BOBA: Byzantine-robust federated learning with label skewness. In *International Conference on Artificial Intelligence and Statistics*, pages 892–900, 2024.

[28] Wenke Huang, Zekun Shi, Mang Ye, He Li, and Bo Du. Self-driven entropy aggregation for byzantine-robust heterogeneous federated learning. In *International Conference on Machine Learning*, pages 20096–20110, 2024.

[29] Shiqi Shen, Shruti Tople, and Prateek Saxena. AUROR: Defending against poisoning attacks in collaborative deep learning systems. In *Annual Computer Security Applications Conference*, 2016.

[30] Clement Fung, Chris J. M. Yoon, and Ivan Beschastnikh. The limitations of federated learning in sybil settings. In *International Symposium on Research in Attacks, Intrusions and Defenses*, 2020.

[31] Zaixi Zhang, Xiaoyu Xu, Jinyuan Jia, Matthew Jagielski, Gauri Joshi, Mingyi Hong, and Carl A. Gunter. FLDetector: Defending federated learning against model poisoning attacks via detecting malicious clients. In *ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2022.

[32] Youssef Allouah, Rachid Guerraoui, Nirupam Gupta, Ahmed Jellouli, Geovani Rizk, and John Stephan. Adaptive gradient clipping for robust federated learning. In *International Conference on Learning Representations*, 2025.

[33] Yi-Rui Yang, Chang-Wei Shi, and Wu-Jun Li. On the tension between byzantine robustness and no-attack accuracy in distributed learning. In *International Conference on Machine Learning*, 2025.

[34] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *IEEE Conference on Computer Vision and Pattern Recognition*, pages 770–778, 2016.

Algorithm 1 CARAT server aggregation at round r

Require: Current global model w_r, client messages $\{u_i^r\}_{i=1}^n$, previous-round weights α^{prev}, probe pool $\mathcal{P}$, CARAT hyperparameters
Ensure: Aggregated update Δ_{agg}, updated temporal prior α^{prev}

```
if P is uninitialized then
    Build a balanced probe pool from the server-visible training dataset
end if
if r = 1 then
    α^prev ← (1/n) 1
end if
for i = 1 to n do
    Convert u_i^r into a vector-form delta Δ_i^r
end for
Compute the clipping threshold τ_r from {‖Δ_i^r‖_2}_{i=1}^n
for i = 1 to n do
    Δ̄_i^r ← min(1, τ_r / (‖Δ_i^r‖_2 + ε)) Δ_i^r
end for
Choose the evaluation radius q_r from a quantile of positive clipped norms
for i = 1 to n do
    Δ̃_i^r ← q_r Δ̄_i^r / (‖Δ̄_i^r‖_2 + ε)
end for
Sample T class-balanced hidden tasks from P with replacement/subset fallback if strict balanced
sampling fails
for t = 1 to T do
    Evaluate the baseline loss ℓ_t(w_r)
end for
for i = 1 to n do
    for t = 1 to T do
        c_{i,t} ← ℓ_t(w_r) − ℓ_t(w_r + Δ̃_i^r)
    end for
end for
Robustly standardize certificate columns to obtain C^std
Compute rank penalties r from repeated coordinate subsamples of {Δ̄_i^r}_{i=1}^n
α^(0) ← α^prev
for k = 0 to K − 1 do
    z ← (C^std)^⊤ α^(k)
    π ← softmin task weights induced by z and β
    g ← C^std π − λ_r r − 2λ_p (α^(k) − α^prev)
    α^(k+1) ← Π_{A(ρ̂)}(α^(k) + ηg)
end for
α* ← α^(K) and α^prev ← α*
Δ_agg ← Σ_{i=1}^n α_i* Δ̄_i^r
return Δ_agg, α^prev
```

A Algorithmic Details

A.1 Round-Level Pseudocode

Algorithm 1 summarizes one CARAT server round in standard pseudocode form.

A.2 Implementation Notes

Inputs. Current global model w_r, client messages $\{u_i^r\}_{i=1}^n$, previous-round weights α^{prev}, server-visible training dataset, and CARAT hyperparameters.

Persistent state. CARAT maintains two pieces of state across rounds: a balanced probe pool sampled at initialization from the server-visible training dataset, and the previous-round weight vector α^{prev}, which serves as the temporal prior for the current round.

Initialization. At the first round, the implementation initializes α^{prev} to the uniform vector. This keeps the initial round unbiased across clients while still allowing later rounds to exploit temporal consistency once CARAT has observed several optimization trajectories.

Reference-pool construction and hidden-task sampling. Let $\mathcal{P}_c \subseteq \mathcal{P}$ denote the subset of probe examples with class label c. A hidden task is constructed by sampling a small number of examples from each available class pool and concatenating them into one balanced micro-task. If the desired number of samples per class is not available without replacement, CARAT falls back to replacement and, if needed, to a random subset of the probe pool rather than aborting the round. This fail-soft design preserves the intended semantic diversity of the task sampler while ensuring that the server always obtains a well-defined certificate matrix.

Robust score calibration. For notational economy, the main text reuses C after standardization. In implementation terms, the raw certificate matrix is first transformed column-wise into

$$C^{\mathrm{std}}_{i,t} = \frac{c_{i,t} - \mathrm{median}_j(c_{j,t})}{1.4826 \cdot \mathrm{MAD}_j(c_{j,t}) + \varepsilon}.$$

This removes task-specific scale effects while preserving within-task ordering by Proposition B.4. Rank penalties are calibrated similarly. For each coordinate subsample S_s, let

$$\rho_i^{(s)} = \frac{1}{|S_s|} \sum_{j \in S_s} \left| \frac{\mathrm{rank}_{i,j}}{n-1} - \frac{1}{2} \right|.$$

CARAT then computes a standardized rank score

$$z_i^{(s)} = \frac{\rho_i^{(s)} - \mathrm{median}_\ell(\rho_\ell^{(s)})}{1.4826 \cdot \mathrm{MAD}_\ell(\rho_\ell^{(s)}) + \varepsilon},$$

and aggregates subsamples through

$$r_i = \max\left(0, \mathrm{median}_s\, z_i^{(s)}\right).$$

As a result, a client is penalized only when it is consistently extreme across repeated coordinate views, not because of a single noisy subsample.

Projected optimization details. Again writing C for the standardized certificate matrix, CARAT initializes the optimizer with $\alpha^{(0)} = \alpha^{\mathrm{prev}}$ after the first round, and with the uniform vector at initialization. If

$$z(\alpha) = C^\top \alpha, \qquad \pi_t(\alpha) = \frac{\exp(-\beta z_t(\alpha))}{\sum_{\ell=1}^{T} \exp(-\beta z_\ell(\alpha))},$$

then the ascent direction used in projected updates is

$$g(\alpha) = C\pi(\alpha) - \lambda_r r - 2\lambda_p(\alpha - \alpha^{\mathrm{prev}}).$$

With optimizer step size η, one iteration takes the form

$$\alpha^{(k+1)} = \Pi_{\mathcal{A}(\hat{\rho})}\left(\alpha^{(k)} + \eta g(\alpha^{(k)})\right).$$

The projection maintains feasibility throughout optimization, so every intermediate iterate already satisfies the nonnegativity, unit-sum, and per-client cap constraints described in the main text.

Round invariants and failure handling. Two implementation choices are important for robustness. First, the optimizer never leaves the capped simplex, so every intermediate and final weight vector remains feasible. Second, probe-task construction is fail-soft rather than fail-stop: if strict class-balanced sampling is impossible, the code falls back to replacement or to a random subset, ensuring that the round can still proceed with a well-defined certificate matrix.

Outputs. Aggregated server update, updated temporal prior α^{prev}, and diagnostic logs including learned weights, task scores, softmin focus, rank penalties, evaluation radius, clipping threshold, and the identities of the hidden probe tasks used in the current round.

Practical diagnostics. The most useful round-level diagnostics for debugging and empirical interpretation are the learned weights, the weakest-task values emphasized by the softmin, the clip threshold τ_r, the evaluation radius q_r, and the final rank penalties r_i. In successful runs, these quantities help distinguish three qualitatively different cases: benign but diverse clients that retain moderate weight, clearly harmful clients that lose certificate support across multiple tasks, and structurally extreme clients whose influence is reduced primarily by the rank penalty.

B Theoretical Analysis

This appendix records the main structural properties used to interpret CARAT. These results are not presented as a complete robustness guarantee against arbitrary adaptive attackers. Instead, they clarify why the method decomposes into a normalized evaluation geometry, a task-aware utility objective, and a constrained robust aggregation step.

Proposition B.1 (Feasibility and support of the capped simplex)**.** *Let*

$$\mathcal{A}(\hat{\rho}) = \left\{ \alpha \in \mathbb{R}^n : \alpha_i \geq 0,\ \sum_{i=1}^{n} \alpha_i = 1,\ \alpha_i \leq \frac{1}{(1-\hat{\rho})n} \right\},$$

with $\hat{\rho} < 1$. Then $\mathcal{A}(\hat{\rho})$ is non-empty, and every $\alpha \in \mathcal{A}(\hat{\rho})$ satisfies

$$|\operatorname{supp}(\alpha)| \geq (1-\hat{\rho})n.$$

Proof sketch. Let $u = 1/((1-\hat{\rho})n)$. Since $\hat{\rho} < 1$, we have $u \geq 1/n$, so the uniform weight vector is feasible and the set is non-empty. For the support bound, if only k coordinates of α are positive, then

$$1 = \sum_{i=1}^{n} \alpha_i \leq ku = \frac{k}{(1-\hat{\rho})n},$$

which implies $k \geq (1-\hat{\rho})n$. Therefore CARAT can suppress suspicious clients, but cannot collapse the aggregate onto an arbitrarily small subset of participants. □

Proposition B.2 (Softmin bounds and gradient form)**.** *Let $s \in \mathbb{R}^T$ and define*

$$\operatorname*{softmin}_{\beta}(s) = -\frac{1}{\beta} \log \left(\frac{1}{T} \sum_{t=1}^{T} e^{-\beta s_t} \right).$$

Then

$$\min_t s_t \leq \operatorname*{softmin}_{\beta}(s) \leq \min_t s_t + \frac{\log T}{\beta}.$$

Moreover,

$$\lim_{\beta \to \infty} \operatorname*{softmin}_{\beta}(s) = \min_t s_t, \qquad \lim_{\beta \to 0} \operatorname*{softmin}_{\beta}(s) = \frac{1}{T} \sum_{t=1}^{T} s_t.$$

Moreover, if $z(\alpha) = C^\top \alpha$ and

$$\pi_t(\alpha) = \frac{\exp(-\beta z_t(\alpha))}{\sum_{\ell=1}^{T} \exp(-\beta z_\ell(\alpha))},$$

then

$$\nabla_\alpha \operatorname*{softmin}_{\beta}(C^\top \alpha) = C\pi(\alpha).$$

Proof sketch. Let $m = \min_t s_t$. Since

$$e^{-\beta m}/T \leq \frac{1}{T} \sum_{t=1}^{T} e^{-\beta s_t} \leq e^{-\beta m},$$

applying $-(1/\beta)\log(\cdot)$ yields

$$m \le \operatorname*{softmin}_{\beta}(s) \le m + \frac{\log T}{\beta}.$$

The limit $\beta \to \infty$ follows immediately from the vanishing gap bound. For $\beta \to 0$, a first-order expansion of the exponential average gives

$$\frac{1}{T}\sum_{t=1}^{T} e^{-\beta s_t} = 1 - \beta \frac{1}{T}\sum_{t=1}^{T} s_t + o(\beta),$$

so the softmin converges to the arithmetic mean. Differentiating the softmin expression with respect to z_t yields the Gibbs weights $\pi_t(\alpha)$, and the chain rule then gives $C\pi(\alpha)$. Hence the ascent direction is a convex combination of certificate columns, with more weight placed on tasks whose current aggregate utility is smaller. □

Proposition B.3 (Directional invariance of certificate scoring)**.** *Suppose two nonzero clipped updates satisfy $\bar{\Delta}_i = a\bar{\Delta}_j$ for some scalar $a > 0$. Then their normalized versions in Eq.* (3) *coincide, i.e.,*

$$\tilde{\Delta}_i = \tilde{\Delta}_j.$$

Consequently, their hidden-task certificates are identical:

$$c_{i,t} = c_{j,t} \qquad \textit{for all } t \in \{1, \dots, T\}.$$

Proof sketch. Eq. (3) rescales every nonzero clipped update to the common radius q_r. Positive collinearity is therefore removed by normalization:

$$q_r \frac{\bar{\Delta}_i}{\|\bar{\Delta}_i\|_2} = q_r \frac{a\bar{\Delta}_j}{\|a\bar{\Delta}_j\|_2} = q_r \frac{\bar{\Delta}_j}{\|\bar{\Delta}_j\|_2}.$$

Substituting the resulting equality into Eq. (4) gives identical task scores. Thus, after clipping, the certificate stage depends on direction rather than residual norm magnitude. □

Proposition B.4 (Task-wise standardization preserves within-task ordering)**.** *Fix a task t and define a standardized certificate score*

$$\hat{c}_{i,t} = \frac{c_{i,t} - m_t}{s_t + \varepsilon},$$

where $m_t \in \mathbb{R}$ and $s_t > 0$ depend only on task t and not on client i. Then for any clients i, j,

$$c_{i,t} \ge c_{j,t} \quad \Longleftrightarrow \quad \hat{c}_{i,t} \ge \hat{c}_{j,t}.$$

Moreover,

$$\hat{c}_{i,t} - \hat{c}_{j,t} = \frac{c_{i,t} - c_{j,t}}{s_t + \varepsilon}.$$

Proof sketch. For a fixed task t, the map $x \mapsto (x - m_t)/(s_t + \varepsilon)$ is affine with strictly positive slope because $s_t + \varepsilon > 0$. Hence it preserves ordering. The difference identity follows by direct subtraction. Therefore column-wise robust standardization calibrates cross-task scale without changing the within-task ranking induced by the raw certificate scores. □

Proposition B.5 (Temporal prior and projected-iterate invariants)**.** *Define*

$$f(\alpha) = \operatorname*{softmin}_{\beta}(C^\top \alpha) - \lambda_r r^\top \alpha - \lambda_p \|\alpha - \alpha^{\text{prev}}\|_2^2.$$

If $\alpha, \alpha' \in \mathcal{A}(\hat{\rho})$ satisfy $C^\top \alpha = C^\top \alpha'$ and $r^\top \alpha = r^\top \alpha'$, then

$$f(\alpha) - f(\alpha') = -\lambda_p \left(\|\alpha - \alpha^{\text{prev}}\|_2^2 - \|\alpha' - \alpha^{\text{prev}}\|_2^2 \right).$$

In addition, under projected gradient ascent

$$\alpha^{(k+1)} = \Pi_{\mathcal{A}(\hat{\rho})}\left(\alpha^{(k)} + \eta \nabla f(\alpha^{(k)}) \right),$$

every iterate remains in $\mathcal{A}(\hat{\rho})$.

Proof sketch. The first statement follows by direct substitution: if the certificate and rank terms match, the only remaining difference in objective value is the quadratic prior term. CARAT therefore prefers, among utility-equivalent feasible weights, the solution closer to α^{prev}. The second statement is immediate from the projection operator. Hence the optimizer preserves three round invariants throughout: $\alpha_i \geq 0$, $\sum_i \alpha_i = 1$, and $\alpha_i \leq 1/((1-\hat{\rho})n)$. □

Proposition B.6 (Norm bound for the final aggregate)**.** *Suppose each clipped update satisfies* $\|\bar{\Delta}_i\|_2 \leq \tau_r$ *and* $\alpha \in \mathcal{A}(\hat{\rho})$*. Then the final aggregate*

$$\Delta_{\text{agg}} = \sum_{i=1}^{n} \alpha_i \bar{\Delta}_i$$

satisfies

$$\|\Delta_{\text{agg}}\|_2 \leq \tau_r.$$

Proof sketch. Because $\alpha_i \geq 0$ and $\sum_i \alpha_i = 1$, the aggregate is a convex combination of clipped updates. By the triangle inequality,

$$\|\Delta_{\text{agg}}\|_2 \leq \sum_{i=1}^{n} \alpha_i \|\bar{\Delta}_i\|_2 \leq \sum_{i=1}^{n} \alpha_i \tau_r = \tau_r.$$

Thus the final server step inherits the clipping radius as a hard norm upper bound. □

Proposition B.7 (Per-round complexity)**.** *Let n be the number of clients, T the number of hidden tasks, m the number of sampled coordinates for rank scoring, S the number of rank subsamples, and K the number of projected-gradient steps. Ignoring constant factors from batch size and model width, the extra defense-side work of CARAT scales as*

$$\textit{probe-model forward passes} + O(Snm + KnT).$$

Proof sketch. CARAT performs one baseline probe evaluation plus n candidate probe evaluations over the T hidden tasks, then computes S rank summaries over m coordinates, and finally executes K projected optimization steps over n weights and T task scores. In the current implementation, the probe-model forward passes dominate wall-clock time, while the rank and optimization stages are lower-order terms controlled by m, S, and K. □

Remark on assumptions. The object $\mathcal{D}^{\text{ref}}$ in the main text denotes the labeled server reference set, while the probe pool $\mathcal{P}$ used during each round is a subsampled working set derived from it. In practice, the server first constructs $\mathcal{P}$ from the full server-visible reference data and then samples hidden tasks from $\mathcal{P}$ at each round. This distinction should remain explicit because it is part of the method's server-side assumption and materially strengthens the setting relative to a fully data-free defense.

C Additional Experimental Details

This appendix records the benchmark design, sensitivity analyses, and reporting scope that support the main-text experimental design.

C.1 Experimental Goals

The empirical study is designed to test properties specific to CARAT rather than to benchmark it as a generic outlier filter. In particular, we ask whether hidden-task validation improves robustness when benign clients are heterogeneous, whether that advantage persists on class-rich benchmarks where class-selective failure matters, and how much robustness is purchased per unit of trusted-data budget and runtime overhead.

C.2 Benchmarks

Primary class-rich benchmark. The main benchmark is CIFAR-100 with ResNet18. This setting best matches CARAT's motivation because the label space is rich enough for different hidden tasks to emphasize different semantic slices, and because average clean accuracy alone can hide substantial per-class degradation.

Scalability benchmark. TinyImageNet with ResNet18 is used to test whether the same aggregation rule remains effective as the number of classes and the visual vocabulary increase. This benchmark is particularly useful for checking whether the hidden-task construction and reference-budget design remain meaningful beyond CIFAR-100.

Application-oriented benchmark. We include CHMNIST with ResNet18 as an application-style benchmark in the appendix. Its role is not to maximize scale, but to show that class-selective failures have a practical interpretation in a domain where server-side auditing with a small labeled reference set is plausible.

Sanity-check benchmark. CIFAR-10 is retained only as a sanity-check benchmark. It remains useful for verifying that CARAT does not fail on standard low-cardinality image classification, but it is not treated as a primary source of evidence because its ten classes are too few to fully exercise the method's class-aware design.

C.3 Threat Models

Untargeted poisoning. The main untargeted suite contains ALIE, FangAttack, MinMax, and MinSum. Together they represent adaptive model-poisoning attacks that aim to remain stealthy under server-side defenses, plus a geometry-aware stressor that is especially informative for Krum-style and history-based defenses.

C.4 Baselines

Untargeted defenses. For untargeted poisoning, the completed primary comparison uses Mean, NormClipping, MultiKrum, FLTrust, FLDetector, TriGuardFL, and CARAT. This set spans plain averaging, norm-based clipping, geometry-based filtering, trusted-data anchoring, historical-trajectory detection, and class-aware hidden-task validation. TrimmedMean remains useful for smaller diagnostic runs, but it is not part of the completed CIFAR-100/ResNet18 non-i.i.d. comparison block.

C.5 Metrics and Fairness

No-attack clean utility is reported separately from attacked runs. The clean metrics are test accuracy and test loss. On class-rich datasets we additionally report macro-average accuracy and worst-class accuracy, because the motivation for CARAT is precisely that averaging can hide semantically concentrated failures. Runtime per round and total defense overhead are reported for all defenses that add nontrivial server-side computation.

Trusted-data fairness is controlled explicitly. CARAT and FLTrust are compared under the same labeled server-reference budget, rather than allowing one method to benefit from more public data. In addition to fixed-budget comparisons, we sweep the reference budget in the appendix to show whether CARAT's advantage comes from the hidden-task validation principle rather than from budget alone.

C.6 Shared Training Protocol

Unless stated otherwise, the completed primary comparison uses 20 clients with full participation, 200 communication rounds, 4 local epochs, and learning rate 0.05. The completed heterogeneity settings are Dirichlet non-i.i.d. partitions with $\alpha \in \{1, 0.5\}$; the more severe $\alpha = 0.1$ CARAT block remains pending and is therefore treated as future completion rather than a main-text result. The main attacker ratio is 20%, while 40% is reserved for appendix stress tests. The target is 3–5 random seeds per configuration, with current CARAT rows reported over available completed seeds. Scaling

studies in the appendix may additionally vary the number of clients or the attacker ratio, but these are treated as stress tests rather than as the core evaluation axis.

C.7 Final Loss Results

Table 3 reports the final test loss values corresponding to the main-text accuracy comparison. It is included in the appendix because it supports the same conclusion as Table 2: CARAT obtains the best average rank under both completed non-i.i.d. regimes.

Table 3: Final test loss under four model-poisoning attacks on CIFAR-100/ResNet18 with 20% malicious clients and non-i.i.d. client partitions. Lower is better. Baseline cells use five completed seeds. CARAT cells use completed incremental runs: $n = 2$ for $\alpha = 1$ ALIE, $n = 1$ for $\alpha = 0.5$ MinSum, and $n = 3$ otherwise.

Dirichlet α	Defense	ALIE	FangAttack	MinMax	MinSum	Avg. rank
1.0	Mean	0.0292 ± 0.0004	0.0294 ± 0.0004	0.0283 ± 0.0004	0.0283 ± 0.0006	3.2
1.0	NormClipping	0.0476 ± 0.0005	0.0501 ± 0.0004	0.0484 ± 0.0004	0.0485 ± 0.0003	7.0
1.0	MultiKrum	0.0361 ± 0.0006	0.0377 ± 0.0005	0.0295 ± 0.0005	0.0301 ± 0.0009	4.8
1.0	FLTrust	0.0409 ± 0.0004	0.0407 ± 0.0006	0.0406 ± 0.0007	0.0406 ± 0.0005	5.8
1.0	FLDetector	0.0305 ± 0.0004	0.0450 ± 0.0035	0.0283 ± 0.0006	0.0284 ± 0.0003	4.0
1.0	TriGuardFL	0.0291 ± 0.0004	0.0280 ± 0.0005	0.0283 ± 0.0004	0.0283 ± 0.0004	2.2
1.0	**CARAT**	**0.0267±0.0009**	**0.0260±0.0005**	**0.0260±0.0007**	**0.0260±0.0010**	**1.0**
0.5	Mean	0.0303 ± 0.0005	0.0302 ± 0.0006	0.0293 ± 0.0001	0.0293 ± 0.0004	3.2
0.5	NormClipping	0.0486 ± 0.0005	0.0517 ± 0.0011	0.0497 ± 0.0005	0.0498 ± 0.0004	7.0
0.5	MultiKrum	0.0382 ± 0.0008	0.0407 ± 0.0010	0.0306 ± 0.0006	0.0308 ± 0.0006	4.8
0.5	FLTrust	0.0420 ± 0.0005	0.0417 ± 0.0005	0.0417 ± 0.0006	0.0417 ± 0.0004	5.8
0.5	FLDetector	0.0319 ± 0.0004	0.0458 ± 0.0011	0.0294 ± 0.0004	0.0292 ± 0.0005	4.0
0.5	TriGuardFL	0.0301 ± 0.0004	0.0290 ± 0.0003	0.0292 ± 0.0004	0.0292 ± 0.0007	2.2
0.5	**CARAT**	**0.0271±0.0006**	**0.0266±0.0010**	**0.0259±0.0003**	**0.0253**	**1.0**

C.8 Codebase-Supported Expansion Matrix

The main-text comparison is intentionally narrower than what the current repository can support. In particular, the codebase already implements a substantially broader set of model-poisoning attacks and defense baselines than the completed CARAT result block uses. Table 4 therefore separates the main-text core suite from the next expansion layers that can be populated incrementally as runs complete. We treat backdoor and data-poisoning implementations as out of scope for the current paper, even though the repository contains them.

C.9 CARAT Hyperparameters, Stress Tests, and Ablation Plan

The current implementation defaults are 8 hidden tasks, 4 probe samples per class, probe batch size 128, probe-radius quantile 0.5, MAD clipping with $k = 2.5$, default attacker prior $\hat{\rho} = 0.2$, certificate temperature $\beta = 12.0$, rank weight 0.20, prior weight 0.05, 4096 sampled coordinates, 5 rank subsamples, 60 projected optimization steps, and optimizer learning rate 0.5.

Two design principles matter when transferring these defaults across datasets. First, the reference budget should scale with the number of classes rather than be fixed as a single dataset-independent total pool size. Second, CARAT should be ablated along the axes that correspond directly to its mechanism: hidden-task count, reference budget per class, rank-penalty weight, temporal-prior weight, and clipping controls. The appendix therefore includes ablations that isolate whether the observed robustness comes from class-balanced hidden tasks, from the structural rank penalty, or simply from access to more trusted data. It also records high-attacker stress tests, scaling analyses, and runtime breakdowns so that robustness, efficiency, and trusted-data dependence are evaluated together.

Table 4: Recommended model-poisoning experiment expansion matrix supported by the current codebase. The "core suite" matches the narrowed main-text story, while the "extension suite" records additional comparisons that the repository can already run.

Axis	**Core suite**	**Codebase-supported extension suite**	**Suggested visualization**
Untargeted attacks	ALIE, FangAttack, MinMax, MinSum	Gaussian, IPM, Mimic, SignFlipping, ThreeDFed	Final robustness tables; round-wise accuracy curves
Untargeted defenses	Mean, NormClipping, MultiKrum, FLTrust, FLDetector, TriGuardFL, CARAT	Median, TrimmedMean, Krum, Bulyan, RFA, DnC, Auror, LASA, FoolsGold, SignGuard, CenteredClipping, Bucketing, SimpleClustering	Grouped comparison tables; per-attack win/loss summaries
Data regimes	Dirichlet non-i.i.d. $\alpha \in \{1, 0.5\}$	$\alpha = 0.1$ completion, i.i.d. diagnostics, high-attacker stress tests (40%), client-count scaling, CIFAR-10 sanity checks, TinyImageNet, CHMNIST	Robustness-vs-heterogeneity curves; benchmark summary tables
Efficiency and fairness	Matched CARAT / FLTrust reference budget; runtime per round	Reference-budget sweeps; runtime breakdowns; trusted-data sensitivity	Overhead bars; budget-response curves; clean-robustness-overhead tradeoff plots
Out-of-scope repo features	–	ModelReplacement, DBA, Neurotoxin, BadNets, EdgeCase, AlterMin, LabelFlipping, FLAME, DeepSight, CRFL	Keep outside current claim set unless the paper scope is widened again